

EPOCH

THE AI-NATIVE CONTROL PLANE FOR SOFTWARE LIFECYCLE & EVOLUTION

NORTH STAR

AI can change your software one task at a time. EPOCH makes sure you do not lose the system in the process.

```
CURRENT WORK -> WEAVE / lifecycle execution
SYSTEM MEMORY -> EPOCH / evolutionary intelligence
FUTURE SPACE -> EPOCH / counterfactual simulation

BOB 2.0 = execution fabric inside the platform
```

Final integrated concept: WEAVE + EPOCH
Prepared for IBM Bob 2.0 Hackathon | September 2026

1. Executive Summary

EPOCH is a platform for a new software-engineering reality: AI agents can now create, modify, test, release and remediate software at a pace that exceeds what human teams can comfortably understand as a whole. The bottleneck is shifting from producing individual changes to maintaining coherence across the sequence of changes.

CORE INSIGHT

A software change can be locally correct while the system trajectory becomes globally worse. EPOCH treats the trajectory - not the individual change - as a first-class engineering object.

The platform merges two complementary layers: WEAVE, the lifecycle control plane that turns requirements, commits, pull requests and incidents into stateful, evidence-producing workflows; and EPOCH, the longitudinal intelligence layer that records those workflows as system mutations, reasons over their cumulative effects, detects architectural and behavioral drift, reconstructs causal chains, and explores counterfactual futures.

Layer	Primary question	Responsibilities
Lifecycle / WEAVE	What are we doing now?	Events, context, plan, delegate, execute, verify, decide, approvals, evidence, replay, incident remediation.
Evolution / EPOCH	What has the system become?	History, mutations, causal chains, drift, phase changes, technical/evolution debt, system trajectory.
Future / EPOCH	What could the system become?	Counterfactual simulations, alternative patches, future-state comparisons, trajectory selection.
Execution / IBM Bob 2.0	How do we perform the work?	Plan/Agent modes, subagents, parallel execution, background work, document understanding, rollback, workflows.

2. The Problem: AI Makes Changes Cheap, Not Coherence

Modern coding agents are increasingly capable of implementing a feature from a natural-language request, navigating repositories, creating tests, changing multiple files and responding to failures. IBM Bob is explicitly positioned as an agentic development platform spanning planning, development, testing, CI/CD and operations, with subagents, parallel execution and reusable workflows.

The difficult question now moves one level up: what happens to the software system after hundreds of AI-generated changes?

Traditional unit of attention	What it captures	What it misses
File	Text and syntax	Cross-file assumptions and long-term impact
Commit	One change	Cumulative effects of previous changes
PR	A reviewable delta	How the delta changes the future trajectory
Test run	Observed behavior for selected scenarios	Unobserved states and long-horizon interactions
Incident	A failure after it occurs	The chain of mutations that made the failure possible
Architecture diagram	A snapshot of intended structure	How the actual system has drifted over time

WHY NOW

SWE-EVO, EvoClaw and RoadmapBench all report strong evidence that frontier coding agents perform materially worse on long-horizon, multi-step software evolution than on isolated coding tasks. See Research Notes for source-backed figures.

3. Why This Is a Real Frontier Problem

The point is not that current agents cannot code. They can. The issue is that isolated coding benchmarks can hide the harder problem of sustaining a coherent system through time.

Source	What it establishes	Why it matters to EPOCH
SWE-EVO (2025)	Long-horizon evolution tasks average ~21 files; GPT-5 + OpenHands resolved 21% vs 65% on SWE-Bench Verified in the cited setup.	Shows a gap between isolated fixes and sustained evolution.
EvoClaw (2026)	Performance drops from >80% on isolated settings to at most 38% in continuous-evolution settings across tested agents/frameworks.	Directly supports the trajectory / accumulated-error thesis.
RoadmapBench (2026)	115 long-horizon version-upgrade tasks across 17 repositories and 5 languages; even Claude Opus 4.7 resolved 39.1% of tasks in the reported evaluation.	Shows that realistic multi-version development remains difficult.
AgenticFlict (2026)	Large dataset of merge conflicts in AI coding-agent PRs; explicitly notes semantic conflicts are not captured by textual merge-conflict detection.	Demonstrates the limits of snapshot / textual conflict reasoning.
IBM Bob V2 (2026)	Background tasks, rollback, document handling, subagents, parallel work and repeatable workflows.	Provides the execution substrate EPOCH can build upon.
IBM July 2026 strategy	Bob is positioned as an agentic SDLC platform with a shared workflow engine, parallel execution, subagents and governance.	WEAVE should extend Bob rather than duplicate it.

4. Product Thesis

THE THESIS

AI made code generation cheap. EPOCH makes continuous software evolution understandable.

EPOCH is not a better code generator, PR reviewer or test generator. Those capabilities are increasingly commoditized and can be supplied by Bob, Claude, Codex, Gemini or open-source coding agents. EPOCH is the persistent system around those models: it remembers how the software changed, interprets what those changes did collectively, detects when the system crosses into a new behavioral or architectural regime, and can use the same agentic execution layer to steer the system toward a better trajectory.

```
CHANGE HAPPENS
↓
WEAVE EXECUTES + CAPTURES EVIDENCE
↓
EPOCH RECORDS SYSTEM MUTATION
↓
TRAJECTORY MODEL UPDATES
↓
DRIFT / PHASE CHANGE / INCIDENT PATTERNS
↓
COUNTERFACTUAL FUTURES
↓
NEXT ENGINEERING WORKFLOW
```

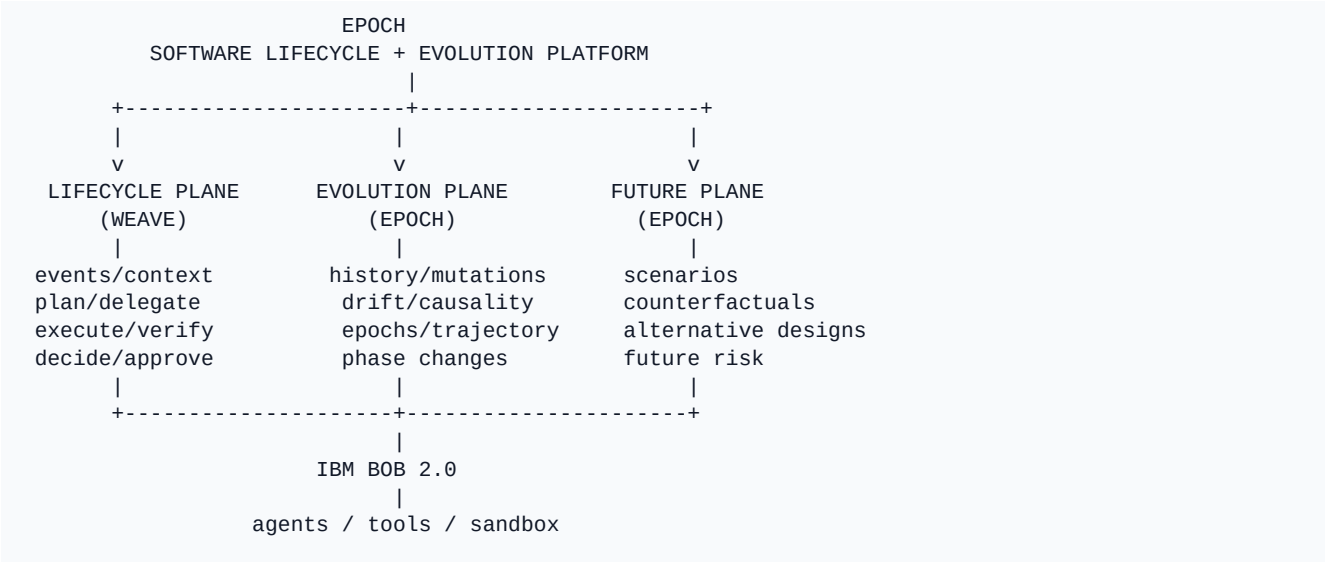
5. The New Objects EPOCH Introduces

Object	Definition	Example
Workflow	A bounded lifecycle execution started by an event.	Implement “30-day chargeback window”.
Mutation	A meaningful change to the system created by one workflow or a set of related actions.	M-1042 changes refund semantics and touches 17 files.
Epoch	A period in which the system exhibits a relatively stable architecture/behavioral regime.	Epoch 04: event-driven payment flow.
Trajectory	The path the system takes across mutations through time.	Coupling grows after five consecutive refactors.
Invariant	A property that should remain true across evolution.	All customer-data access passes through CustomerService.
Evidence	An observable artifact supporting a claim about the system.	Test result, runtime trace, dependency graph, incident reproduction.
Drift	Growing divergence between observed system properties and intended properties.	Direct database access bypasses the intended boundary.
Phase transition	A structural change large enough to make the system meaningfully different.	Monolith → event-driven architecture.
Counterfactual	A plausible alternative future generated by changing a mutation or decision.	Alternative design that preserves invariants with less coupling.

THE ABSTRACTION SHIFT

A commit is an event. A mutation is a consequence. An epoch is a regime. A trajectory is the system history interpreted as a whole.

6. Unified Architecture: WEAVE Inside EPOCH



WEAVE remains a complete lifecycle control plane. EPOCH adds the missing longitudinal model. The merge is not “two products on one dashboard”; every lifecycle outcome becomes a mutation observation, and every evolutionary finding can initiate a new lifecycle workflow.

7. WEAVE Layer: What We Preserve in Full

Nothing important from the WEAVE design is removed. Its components become the operational foundation of EPOCH.

WEAVE component	Retained behavior	Role inside EPOCH
Event intake	PRD, commit, PR, release signal or runtime alert starts a workflow.	Produces the event that becomes the beginning of a mutation record.
Context bundle	Repository, requirements, tests, config and telemetry with provenance.	Provides both local execution context and historical evidence.
Plan	Bob decomposes the requested work and acceptance criteria.	Defines intended mutation before execution.
Specialist agents	Security, QA, release, incident, adversarial and synthesis roles.	Produces structured evidence used immediately and longitudinally.
Parallel execution	Independent investigations run concurrently.	Reduces wall-clock time and creates richer evidence.
Background work	Long-running tasks continue without blocking the developer.	Supports evolution analyses and simulations.
Sandbox	Isolated code/test/build execution.	Enables safe mutation experiments and future simulations.
Verification	Results compared with requirements and checks.	Closes immediate workflow state and feeds evidence to EPOCH.
Approval gates	Consequential changes stop for human/policy approval.	Preserves human control over evolutionary direction.
Rollback	Restore a prior file/task state.	Supports controlled experiments and trajectory correction.
Replay / audit	Chronological lifecycle history.	Becomes the raw temporal history used by the evolution model.
Incident remediation	Alert → reproduction → patch → tests → approval.	Provides the bridge from runtime failures back into system evolution.

8. EPOCH Layer: What Is Added Above WEAVE

EPOCH capability	What it does	Output
Evolution graph	Connects workflows, changes, components, runtime events and decisions over time.	Queryable longitudinal system model.
Mutation engine	Converts lifecycle outcomes into structured system mutations.	Mutation records with scope and consequences.
Trajectory engine	Tracks how architecture/behavior changes across mutations.	Trend and trajectory state.
Drift detector	Compares actual system behavior/structure with intended invariants.	Drift findings with evidence.
Phase detector	Finds structural regime changes that define a new epoch.	Epoch boundary proposal.
Causal archaeology	Traces current symptoms backward through mutation ancestry.	Candidate causal chain and earliest responsible mutation.
Counterfactual simulator	Tests alternative changes or design choices in isolated futures.	Future trajectories for comparison.
Evolution debt model	Tracks accumulated divergence from intended design direction.	Actionable debt / risk map.
Trajectory-aware rollback	Selects prior states or alternative mutations that restore stability.	Safer restoration path.
Future planner	Uses scenario results to propose the next lifecycle action.	New WEAVE workflow.

9. Unified Data & Event Model

The platform needs a common vocabulary so lifecycle execution and long-term evolution can reference each other without losing provenance.

Entity	Minimum fields	Why it exists
Event	event_id, type, source, timestamp, repo, branch, payload	Immutable trigger record.
Workflow	workflow_id, trigger_event, owner, status, current_stage, timestamps	Stateful lifecycle identity.
Task	task_id, agent_type, inputs, dependencies, status, retry_count, output_refs	Unit of specialist work.
Artifact	artifact_id, type, source_task, content_ref, hash, created_at	Traceable output.
Evidence	evidence_id, claim/finding, source_artifact, status, provenance	Separates observation from interpretation.
Decision	decision_id, approver/policy, action, scope, rationale, timestamp	Human/policy accountability.
Mutation	mutation_id, workflow_id, affected_nodes, delta, intent, evidence_refs	Bridge from workflow to system evolution.
Invariant	invariant_id, statement, owner, evidence_refs, validity	Represents intended properties.
Epoch	epoch_id, start, end, defining_properties, boundary_evidence	Represents stable system regimes.
TrajectoryPoint	timestamp, state_hash, structure, behavior_features, evidence_refs	System state over time.
Incident	incident_id, signal, affected_component, reproduction, remediation_ref	Closes runtime back into engineering.
Simulation	simulation_id, base_state, hypothesis, changes, outcome, evidence	Counterfactual future.

INVARIANT DESIGN

The system should distinguish observed facts, model inference and proposed explanation. EPOCH can suggest causality; the prototype should not claim mathematical proof of causality.

10. The Two-Speed Control Loop

Time scale	State machine	Purpose
Micro / workflow	Observe → Understand → Plan → Delegate → Execute → Verify → Decide	Complete the current engineering task.
Macro / evolution	Stable → Mutating → Drifting → Transition → New Epoch	Understand the long-term state of the software system.
Future / simulation	Hypothesis → Fork → Execute → Compare → Select	Explore alternate trajectories before committing to a consequential change.

```
MICRO LOOP
event -> context -> plan -> agent work -> evidence -> decision
      |
      v
    MUTATION RECORD
      |
      v
MACRO LOOP
history -> trajectory -> drift -> phase change -> future scenarios
      |
      v
    NEXT WORKFLOW
```

11. End-to-End Platform Flow

1. A requirement, pull request, release signal or incident enters EPOCH.
2. WEAVE constructs a scoped context bundle with provenance.
3. Bob Plan mode converts the event into an explicit lifecycle plan and task graph.
4. Bob delegates independent work to narrow specialist agents.
5. Agents inspect, modify, test, simulate or investigate inside a sandbox.
6. Verification produces structured evidence and unresolved issues.
7. A human or policy gate decides whether the consequential action can proceed.
8. The completed workflow is committed to the evolution graph as a mutation.
9. EPOCH recalculates system state, drift, dependencies and trajectory.
10. If the mutation creates concerning trajectory divergence, EPOCH launches counterfactual simulations.
11. The chosen corrective path becomes a new WEAVE workflow.
12. The loop continues, producing a continuously evolving system model.

12. Multi-Agent System Design

Agent	Primary input	Output / contract	Guardrail
Historian	Git history, PRs, ADRs, prior workflows	Relevant history and context	Read-focused; source provenance required.
Context / Requirements	PRD, issue, docs, policy	Structured requirement + acceptance criteria	Must surface missing context.
Security	Diff, dependencies, policy	Findings, evidence, severity	No silent approval.
QA	Requirement, diff, existing tests	Tests, mocks, results, coverage evidence	Reproducible outputs required.
Release	Validated change, env config	Build/deploy artifacts	Stops at policy gate.
Incident	Alert, logs, recent changes	Reproduction, candidate cause, patch plan	Candidate explanation, not asserted certainty.
Evolution Analyst	Mutation history, architecture, runtime signals	Drift and trajectory analysis	Must separate observation from inference.
Counterfactual Engineer	Proposed mutation + baseline	Alternative futures	Sandbox only.
Adversarial Maintainer	Current trajectory + assumptions	Failure paths, counterexamples	Goal is falsification, not confirmation.
Synthesis / Bob	All specialist evidence	Next lifecycle action + review package	Preserve source evidence.

13. IBM Bob 2.0: Execution Fabric, Not a Logo

IBM Bob V2 is a natural execution substrate for the merged platform. IBM documents Plan/Agent modes, background tasks, rollback, document understanding, subagents and repeatable workflows. IBM also positions a shared workflow engine as a foundation for governed multi-step engineering work across the software lifecycle.

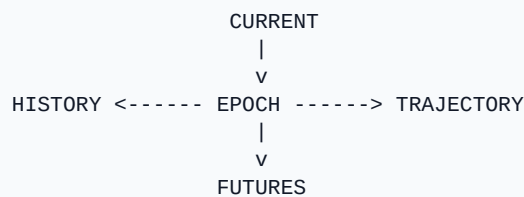
Bob capability	EPOCH use	Judge-visible proof
Plan mode	Compile requirement/incident into lifecycle plan and acceptance criteria.	Plan graph shown before execution.
Agent mode	Implement or modify the proposed mutation.	Repository diff linked to workflow.
Subagents	Run specialized QA/security/history/evolution work.	Independent task cards and evidence.
Parallel execution	Run independent analyses concurrently.	Concurrent task timestamps.
Background tasks	Long-running trajectory analyses and simulations.	Simulation remains active while console is usable.
Documents	Read PRDs, ADRs, PDFs and other non-code context.	Context references shown with provenance.
Rollback	Experiment on branches and restore prior states.	Reversible mutation path.
Workflows	Package repeatable lifecycle patterns.	Same golden path replayed on clean run.
MCP / tools / hooks	Connect repository, CI and policy actions.	Explicit tool boundaries and event logs.

STRATEGIC FIT

Do not compete with Bob on code generation. Make Bob more valuable by giving its actions a longitudinal system context and a model of what the software is becoming.

14. Product Experience: One Console, Four Lenses

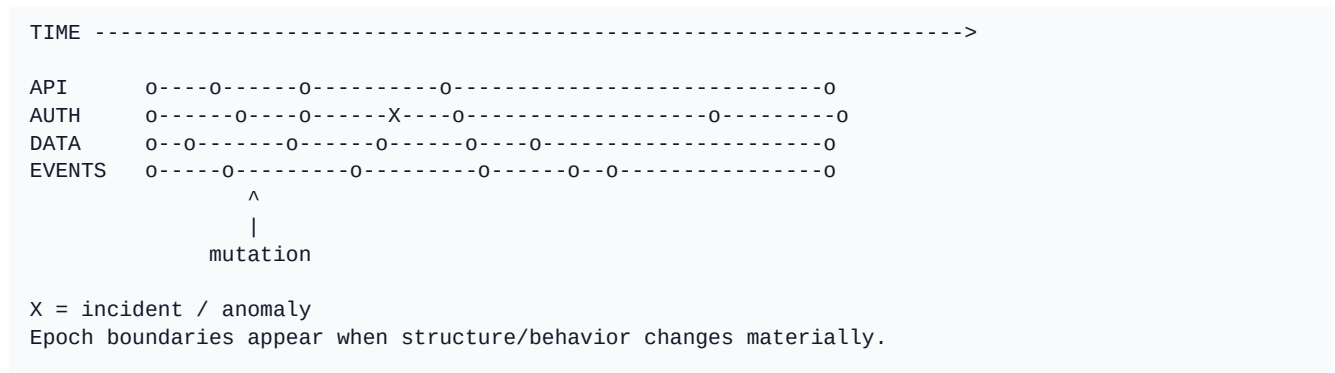
Lens	User question	Primary view
CURRENT	What is happening now?	Lifecycle timeline, active agents, current evidence, next gate.
HISTORY	How did we get here?	Mutations, commits, incidents, decisions and replay.
TRAJECTORY	What is the system becoming?	Evolution graph, drift, invariants, architecture/behavior changes.
FUTURES	What could happen next?	Counterfactual branches, scenario comparison, recommended next move.



The console should feel like a mission control system for a living software system, not a pile of chat windows. The developer interacts primarily with decisions and evidence; specialist-agent activity remains inspectable but secondary.

15. Evolution Graph

The most distinctive visual object is a time-aware graph in which software components, mutations, incidents, invariants and decisions are connected along the system timeline.



- Click any mutation to open the exact WEAVE workflow that created it.
- Click any incident to trace the likely upstream mutation chain.
- Click any invariant to see which changes strengthened or weakened it.
- Drag across time to compare two system states.
- Fork from a mutation to launch a counterfactual future.

16. Causal Archaeology

Causal archaeology is not “AI magically proving causality.” It is a disciplined process for reconstructing a candidate explanation from observed evidence and temporal relationships.

```

CURRENT SYMPTOM
  |
  v
AFFECTED COMPONENT
  |
  v
RECENT MUTATIONS
  |
  v
DEPENDENCY / BEHAVIOR LINKS
  |
  v
EARLIEST PLAUSIBLE MUTATION
  |
  v
COUNTEREXAMPLE / REPRODUCTION
  |
  v
CANDIDATE EXPLANATION + EVIDENCE

```

LANGUAGE DISCIPLINE

Use “candidate causal chain” or “earliest plausible mutation” in the prototype. Reserve stronger causal claims for situations where the evidence genuinely supports them.

17. Trajectory Drift & Phase Changes

A major EPOCH feature is the ability to detect a sequence of individually acceptable mutations that collectively move the system away from an intended design direction.

Signal	Example	Interpretation
Boundary erosion	More services access a database directly.	Intended architecture is weakening.
Invariant erosion	Exceptions accumulate around one business rule.	The original business invariant is becoming ambiguous.
Dependency growth	A service gains many cross-domain dependencies.	Coupling trajectory is rising.
Behavioral change	Retry volume, latency or error shape changes after several mutations.	The system has entered a new operating regime.
Semantic conflict	Two changes are both syntactically valid but disagree on behavior.	Snapshot-based review is insufficient.
Agentic accumulation	Different agents make compatible-looking changes that create long-term inconsistency.	Need longitudinal coordination and memory.

PHASE TRANSITION

The platform should propose a new epoch when structural or behavioral properties become sufficiently different from the prior regime. The threshold is a configurable research/engineering heuristic in the prototype.

18. The Mutation Record

```
M-1042
Intent: Extend chargeback eligibility from 15d -> 30d

Touched components: API | Order Service | DB | Archival Job
Evidence: tests | diff | runtime trace | requirement
Immediate effect: API behavior correct
Structural effect: retention path still 15d
Downstream mutations: M-1051, M-1077
Incident relation: INC-3312
Trajectory effect: invariant weakened
```

The mutation is not simply a commit hash. It is a compact statement of intent, scope, evidence, immediate result and observed downstream consequences.

19. Counterfactual Futures

When the platform detects a risky trajectory, it can fork the current system into isolated candidate futures. The point is not to predict the future with certainty; it is to compare plausible engineering choices under consistent experiments.

Scenario	Change	Likely benefit	Likely cost / risk
A - Keep current plan	Apply proposed mutation as-is	Fastest completion	Higher coupling
B - Refactor boundary	Move shared logic behind one service	Restores architecture	More files / larger short-term change
C - Compatibility layer	Preserve old interface while adding new behavior	Lower migration risk	Temporary complexity

DECISION RULE

EPOCH should surface evidence and trade-offs. A human or policy gate decides which trajectory to adopt.

20. Evolution Debt

Traditional technical debt is often a list of known shortcuts. EPOCH introduces a broader operational concept: evolution debt is the accumulated divergence between the system being built and the system the organization intends to operate.

Dimension	Example indicator	What EPOCH does
Architecture	More bypass paths over time	Shows mutation chain behind each bypass.
Business rules	Duplicated or contradictory enforcement	Finds where exceptions accumulated.
Dependencies	Increasing cross-service coupling	Visualizes the growth trajectory.
Runtime behavior	Slowly rising retries or tail latency	Links behavioral changes to mutation history.
Knowledge	Key assumptions no longer reflected in docs	Surfaces stale or contradicted context.
Agentic drift	Repeated small AI changes produce structural divergence	Groups mutations into patterns rather than reviewing one-by-one.

21. Incident-to-Evolution Closed Loop

```
RUNTIME INCIDENT
|
v
WEAVE INCIDENT WORKFLOW
|
+--> Reproduce
+--> Inspect logs / telemetry
+--> Trace recent mutations
+--> Propose patch
+--> Validate
+--> Human approval
|
v
EPOCH UPDATES TRAJECTORY
|
+--> local defect
+--> systemic drift
+--> new invariant needed
+--> new epoch proposed
```

The same platform handles forward delivery and backward remediation. This closes one of the most important seams in the SDLC: production failures do not start an entirely separate investigation; they re-enter the same historical and lifecycle context that created the system.

22. Safety, Governance & Boundaries

Boundary	Design rule
Model fallibility	AI outputs are proposals/inferences, not unquestionable truth.
Sensitive repositories	Least privilege, sandboxing and explicit approvals.
Production actions	Never autonomous in the prototype; stop at human/policy gate.
Evidence	Preserve source references; distinguish observation from interpretation.
Retries	Bound retries and escalate rather than looping indefinitely.
Simulation	Counterfactual work runs on isolated branches/snapshots.
Causality	Use evidence-backed candidate chains, not unsupported certainty.
Audit	Every lifecycle decision and mutation remains replayable.

23. Golden Demo: The First-Place Story

The demo must make the conceptual leap visible. Do not start by showing an agent writing code. Start by showing that multiple locally-correct changes can make a system globally worse.

1. Open a realistic payment application with a seeded history of ~20-30 prior changes and a few architecture decisions.
2. Show the EPOCH trajectory view: current state, prior epochs and recent mutations.
3. Submit a natural-language requirement through WEAVE: extend chargeback eligibility from 15 to 30 days.
4. Bob enters Plan mode; Historian, Security and QA inspect context while an evolution analysis computes the likely trajectory impact.
5. Bob implements the change. Parallel specialists validate it and the workflow reaches an approval gate.
6. Replay a sequence of additional safe-looking AI changes. Each one passes local tests.
7. EPOCH detects a trajectory divergence: the system is accumulating direct data-access paths that weaken an intended boundary.
8. Click “Why?” and show the mutation chain. Identify the earliest plausible mutation.
9. Launch two counterfactual futures in parallel: preserve the current path or repair the architecture while keeping the valid feature changes.
10. Select the safer future and send it into a new WEAVE remediation workflow.
11. Run verification, human approval and commit the repaired mutation.
12. Return to the trajectory view and show the system back inside the intended envelope.

DEMO LINE

A change can be correct. A system can still be getting worse.

24. Judge-Facing UX Storyboard

Moment	Screen	What the judge should understand
0:00	System overview	This is a living software system, not a code editor.
0:20	WEAVE workflow	Bob coordinates real work with specialist agents.
0:50	Mutation created	This workflow becomes part of the system history.
1:30	Trajectory graph	EPOCH sees the cumulative system, not one PR.
2:00	Drift alert	Locally-correct changes can collectively move the system off-course.
2:30	Causal archaeology	We can trace back to the earliest plausible mutation.
3:00	Counterfactuals	We can explore alternative futures before committing.
3:40	Remediation	Bob/WEAVE executes the selected corrective workflow.
4:20	Closed trajectory	The system returns to a stable intended regime.
4:40	Final statement	EPOCH manages the evolution of AI-built software.

25. Two-Day MVP: What We Actually Build

PROTOTYPE PRINCIPLE

Prove one vertical slice of the platform deeply. Do not build enterprise integrations. The architecture should be broader than the implementation.

Module	Two-day prototype	Not required
Sample project	One realistic payment/e-commerce repository with seeded history.	Large enterprise repo.
WEAVE control plane	Event intake, task graph, specialist agents, evidence, approval.	Universal workflow authoring.
Evolution graph	Commit/workflow/mutation timeline with component links.	Full repository-scale graph analytics.
Drift detector	Two or three deterministic drift patterns.	General theorem of architecture health.
Causal archaeology	Graph traversal + evidence ranking.	Mathematical causal proof.
Counterfactuals	Two isolated candidate branches.	Large-scale model predictive simulation.
Bob integration	Plan/Agent/subagents/background/rollback/workflow features used in the golden path.	Every Bob capability.
Console	Current / History / Trajectory / Futures views.	Production UI system.
Metrics	Wall-clock, manual steps, checks, detected drift, remediation time.	Statistically significant enterprise benchmark.

26. 36-Hour Build Plan

Time	Focus	Hard gate
0-4h	Repo + event schema + state store + Bob connection	One event reaches a workflow.
4-9h	WEAVE lifecycle state machine + console skeleton	Requirement → plan → task graph visible.
9-15h	Specialist agents + parallel evidence capture	Security/QA/history return structured evidence.
15-20h	Mutation recorder + evolution graph	Every completed workflow appears as a mutation.
20-25h	Drift + phase detection	Seeded trajectory divergence is detected.
25-29h	Causal archaeology + counterfactual branch	Earliest plausible mutation and two futures visible.
29-33h	Closed-loop remediation	Chosen future re-enters WEAVE and verifies.
33-36h	Polish + clean rerun + video	Golden demo survives a clean restart.

27. Evaluation & Proof of Impact

The challenge asks for productivity, reduced effort, reduced errors/rework or reduced task time. Measure the prototype instead of inventing percentage claims.

Metric	Baseline	EPOCH measurement
Workflow setup time	Manual coordination across tools	Time from requirement to active plan
Validation coordination	Manual sequential checks	Elapsed time and engineer actions with parallel agents
Incident-to-remediation	Separate investigation workflow	Time from incident to tested remediation package
Manual handoffs	Count of tool/context transitions	Workflow event log count
Drift detection latency	Human discovery after incident	Time from first trajectory divergence to EPOCH alert
Causal search effort	Developer search through history	Time/steps to retrieve candidate mutation chain
Simulation usefulness	One implementation path	Number of candidate futures compared before decision
Replayability	Reconstruct manually	Successful clean rerun from fixed seed

28. Why This Is Not “Another AI Coding Assistant”

Category	Typical center of gravity	EPOCH center of gravity
Code generation	Produce code faster	Consume changes as mutations in a persistent system model.
PR review	Evaluate a diff	Evaluate how a diff affects the long-term trajectory.
Testing	Run or generate tests	Use tests as evidence about system evolution.
Debugging	Find current bug	Trace current behavior back through mutation ancestry.
Architecture tools	Show current structure	Model structural change over time and detect phase changes.
Agent orchestration	Coordinate current agents	Coordinate work while maintaining evolutionary memory.
CI/CD	Move code through gates	Use releases as observations of the system trajectory.

POSITIONING

WEAVE is the execution/control plane. EPOCH is the longitudinal intelligence layer. Bob is the execution fabric. The product is the combination.

29. Defensibility & Platform Moat

The moat is not the UI and not a better prompt. The moat is the persistent model that accumulates with every lifecycle event.

Asset	Why it compounds
Mutation history	Each workflow becomes structured knowledge about how a real system changes.
Causal patterns	Repeated incidents reveal common ancestry and repair patterns.
Trajectory baselines	The platform learns what healthy evolution looks like for a system/team.
Invariant library	Organizations accumulate machine-readable statements of what must remain true.
Counterfactual outcomes	The platform learns which alternative changes actually worked.
Evolution fingerprints	Different systems develop recognizable trajectories and failure signatures.
Agent behavior memory	The platform can learn where agent-generated changes tend to create downstream problems.

Long-term, EPOCH becomes an institutional memory for AI-native software engineering: not only “what code exists,” but “why it became this way, what has already been tried, what failed, and what trajectories remain safe.”

30. Enterprise / Platform Opportunity

Customer need	EPOCH value
Organizations using many coding agents	Longitudinal visibility into system evolution.
Large legacy modernization programs	Historical context plus mutation-aware planning.
Highly regulated engineering	Replayable lifecycle decisions and evidence.
Large software portfolios	Cross-repository evolution patterns and common failure signatures.
AI governance teams	Understand what autonomous development is changing over time.
Platform engineering teams	Detect architectural drift before incidents.
Engineering leadership	See whether faster development is producing healthier or less coherent systems.

Potential future product surfaces: enterprise policy packs, organization-wide evolution maps, historical failure analytics, agent behavior analytics, modernization trajectories, architecture health services and APIs for external CI/CD/observability systems.

31. Expansion Roadmap

Phase	Capability	Strategic reason
Hackathon	One repo, one golden path, mutation graph, drift, causal archaeology, counterfactuals.	Prove the thesis.
Team	More repositories, reusable lifecycle templates, richer invariants and policy.	Make lifecycle intelligence repeatable.
Enterprise	RBAC, identity, secrets boundaries, CI/CD and observability adapters.	Make the control plane deployable.
Intelligence	Historical trajectory learning, recurring drift patterns, predictive maintenance of architecture.	Turn history into compound intelligence.
Platform	Cross-repository evolution graph, agent marketplace, organization-wide policy, external APIs.	Become system-of-systems infrastructure.

32. Risks, Limitations & Honest Boundaries

Risk	Why it matters	Response
Incomplete context	Agents can reason incorrectly when context is missing.	Show provenance; surface missing context; never silently guess.
Model error	Generated code and explanations can be wrong.	Evidence, reproducible tests and human gates.
Causal overreach	Temporal correlation is not proof of causality.	Use candidate chains and evidence-backed hypotheses.
State explosion	Longitudinal system models can become large.	Scope graph to relevant components and use hierarchical summaries.
Simulation fidelity	A counterfactual branch may not perfectly represent production.	Label simulations as scenarios; use them comparatively, not as proof.
Compute cost	Parallel agents and simulations can be expensive.	Bound fan-out and use targeted triggers.
Enterprise integration	Real environments differ widely.	Adapters and narrow initial scope.
Hackathon maturity	Prototype is not a production control plane.	Keep claims disciplined.

33. Judge / Rubric Strategy

The competition brief emphasizes developer workflow improvement, clear productivity impact, use of Bob 2.0 features beyond simple coding assistance, and working end-to-end demonstration. The merged EPOCH architecture maps directly to that requirement.

Judging dimension	How EPOCH answers it
Technology usage	Bob is the execution fabric: Plan/Agent, subagents, parallel work, background tasks, documents, rollback and workflows are all used in the same golden path.
Presentation	The story is visual: current workflow → mutations → trajectory divergence → causal archaeology → counterfactual future → repaired trajectory.
Business value	Reduces coordination cost now and addresses a larger enterprise problem: maintaining coherent software as AI increases change velocity.
Originality	The central object is the evolving software trajectory, not the individual code change. WEAVE provides execution; EPOCH provides longitudinal intelligence.

FIRST-PLACE POSITIONING

The project should never be introduced as “an AI developer assistant.” Introduce it as the control-and-intelligence plane for software that never stops changing.

34. Hard Questions Judges May Ask

Q: Is this just another code-review tool?

A: No. Code review evaluates a delta. EPOCH evaluates the system trajectory that emerges from many deltas and can trace a current problem back through mutation history.

Q: Why can't Claude/Codex/Bob simply do this?

A: They can perform many constituent tasks. The differentiated layer is persistent system history, mutation relationships, trajectory analysis, phase detection and counterfactual state management across workflows.

Q: Why not just use Git history?

A: Git provides raw history. EPOCH turns history into a structured evolutionary model linking intent, execution, evidence, runtime behavior, incidents and future scenarios.

Q: Isn't architecture drift already known?

A: Yes. Architectural drift is known; the proposed contribution is a longitudinal, agent-native system that connects drift to the exact workflow mutations that created it and can route corrective action through the same lifecycle control plane.

Q: Can you really prove causality?

A: Not in the absolute sense. The prototype produces candidate causal chains ranked by evidence and temporal/dependency relationships, and it explicitly labels inference vs observed evidence.

Q: What is the first practical use case?

A: AI-native application maintenance: keep a real application coherent while multiple agent-generated changes accumulate over time.

Q: What happens if EPOCH itself is wrong?

A: It is advisory. Consequential changes remain behind human/policy gates. The system exposes evidence and uncertainty rather than silently enforcing speculative conclusions.

Q: Why does IBM benefit?

A: EPOCH extends Bob from a powerful execution fabric into a longitudinal intelligence layer for AI-native engineering, making Bob more valuable as the pace of autonomous development increases.

35. Final Pitch Narrative

OPENING

AI has made software changes cheap. But software is not a collection of changes. It is a living system with memory.

1. Today, coding agents can implement features, fix bugs and modernize code at extraordinary speed.
2. The unresolved problem is long-horizon coherence: dozens or hundreds of locally-correct changes can accumulate into architectural drift, semantic conflicts, hidden dependencies and new failure modes.
3. WEAVE gives EPOCH the lifecycle control plane: it turns requirements, PRs, releases and incidents into stateful, evidence-producing workflows.
4. EPOCH gives WEAVE longitudinal intelligence: it records each workflow as a system mutation, models trajectories, detects drift and phase changes, and reconstructs how the current state emerged.
5. When the trajectory becomes concerning, EPOCH forks counterfactual futures and uses Bob to execute the selected corrective path through the same governed lifecycle.
6. The result is a system that does not merely help AI write software; it helps organizations understand and steer what AI-written software becomes.

CLOSING LINE

AI can change your software one task at a time. EPOCH makes sure you do not lose the system in the process.

36. Research & Source Notes

The following sources informed the final concept. Current product claims are attributed to IBM/primary sources; benchmark figures are reproduced only where explicitly stated by the source.

R1. IBM Bob V2: Faster, better, smarter (June 24, 2026).

<https://bob.ibm.com/blog/bob-v2-release-announcement/>

Bob V2 architecture, background tasks, rollback, document handling, subagents and workflows.

R2. IBM: Bob advances agentic software development with Premium Packages and a new enterprise AI foundation (July 9, 2026).

<https://www.ibm.com/new/announcements/ibm-bob-expands-with-premium-packages-new-architecture-and-greater-enterprise-control>

Shared workflow engine, parallel execution, subagents, background task orchestration and enterprise lifecycle framing.

R3. IBM Newsroom: Multi-Agent Capabilities and Specialized Modernization Workflows (July 9, 2026).

<https://newsroom.ibm.com/2026-07-09-ibm-advances-enterprise-ai-software-development-with-multi-agent-capabilities-and-specialized-modernization-workflows>

IBM statement that AI is shifting bottlenecks toward review and validation; broader SDLC direction.

R4. SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios (2025).

<https://arxiv.org/abs/2512.18470>

Evidence that sustained multi-file evolution is substantially harder than isolated coding.

R5. EvoClaw: Evaluating AI Agents on Continuous Software Evolution (2026).

<https://arxiv.org/abs/2603.13428>

Evidence of large performance drops in continuous evolution settings.

R6. RoadmapBench: Evaluating Long-Horizon Agentic Software Development Across Version Upgrades (2026).

<https://arxiv.org/abs/2605.15846>

Multi-version, multi-file long-horizon benchmark and frontier-model results.

R7. AgenticFlict: Large-Scale Dataset of Merge Conflicts in AI Coding Agent Pull Requests on GitHub (2026).

<https://github.com/unlv-evol/AgenticFlict>

Evidence and limits around conflicts in AI coding-agent PRs; semantic conflicts remain out of scope for textual merge detection.

R8. IBM Bob developer tutorials (2026).

<https://developer.ibm.com/components/ibm-bob/tutorials/>

Examples of Bob workflows and integrations in modernization and agentic development.

R9. IBM Bob: Your editor, your policies, your audit trail (August 2026).

<https://bob.ibm.com/blog/august-2026-release-2/>

Hooks, deterministic rails around agent loops and enterprise policy/audit framing.

37. One-Page Product Definition

Field	Definition
Product	EPOCH
Category	AI-native software lifecycle and evolution control plane
Core insight	Individual changes can be locally correct while the system trajectory becomes globally worse.
WEAVE role	Operational lifecycle control plane for current work.
EPOCH role	Longitudinal intelligence layer for system evolution.
Bob role	Execution fabric for planning, delegation, implementation, validation and controlled change.
Primary users	Software engineers, platform teams, engineering leaders, modernization teams.
First wedge	AI-native application maintenance and long-horizon evolution.
Signature visual	Evolution graph across time with mutation, drift, incident and epoch overlays.
Signature moment	“A change can be correct. A system can still be getting worse.”
Signature workflow	Detect trajectory divergence → trace mutation ancestry → simulate futures → execute corrective workflow → re-evaluate trajectory.
Long-term moat	Persistent mutation history, trajectory baselines, causal patterns, invariants and counterfactual outcomes.

FINAL DEFINITION

EPOCH is an AI-native control and intelligence plane for software that never stops changing: WEAVE coordinates the work happening now; EPOCH remembers, interprets and simulates the evolution created by that work; IBM Bob 2.0 provides the agentic execution fabric.